¿TU APP EXPONE TUS CLAVES?

checklist de seguridad

Una revisión práctica para saber si alguien puede ver tus claves de OpenAI, Supabase o Stripe, y qué hacer si ya están expuestas. Pensada para apps hechas con IA o con vibe coding.

01 Revisa en 5 minutos
02 Tu repositorio
03 Por servicio
04 Protecciones extra
05 Si encontraste una
06 ¿Lo revisamos juntos?

PARTE 01

REVISA EN 5 MINUTOS

Primero: ¿qué es una clave expuesta?

Es una clave secreta que cualquiera puede ver desde el navegador o desde tu GitHub. Con ella pueden gastar tu saldo de IA, leer o borrar tu base de datos o usar tu cuenta de pagos. Pasa mucho en apps hechas con IA: el asistente pone la clave donde funciona, no donde es segura.

Abre tu app publicada → clic derecho → Inspeccionar

Sources

Busca palabras clave en el código

Pulsa Ctrl/Cmd + Shift + F y busca: `sk-` `sk_live` `sk-ant-` `service_role` `secret` `api_key` `AIza`. Si aparece una clave secreta, está expuesta.

Network

Mira a quién llama tu app

Usa la app con esta pestaña abierta. Si ves llamadas directas a `api.openai.com`, `api.anthropic.com` o `api.stripe.com` con una clave en los headers, está expuesta.

Variables

Revisa los prefijos públicos

Todo lo que empieza por `NEXT_PUBLIC_` `VITE_` `REACT_APP_` o `EXPO_PUBLIC_` se mete en el código del navegador. Ahí solo van claves públicas.

Marca lo que ya revisaste

- [ ] Busqué las palabras clave en Sources y no apareció ninguna clave secreta.
- [ ] Ninguna petición en Network lleva una clave secreta.
- [ ] Mis variables con prefijo público solo tienen claves públicas.

PARTE 02

TU REPOSITORIO

Si tu código está en GitHub, la clave puede estar ahí aunque ya no la veas en el proyecto. Borrar un archivo no lo quita del historial.

- [ ] El archivo `.env` está en `.gitignore` y no aparece en GitHub.
- [ ] El `.env` nunca se subió antes. Compruébalo con `git log --all -- .env`: si sale algo, se subió.
- [ ] Ninguna clave está escrita directo en el código, como `const apiKey = "sk-..."`.
- [ ] Si el repositorio es público, sé exactamente por qué. Si no hace falta, lo pongo privado.
- [ ] Tengo activado **Secret scanning** y **Push protection** en GitHub (Settings → Code security). Bloquean el push si subo una clave.
- [ ] Revisé todo el historial de una vez con `gitleaks detect` o `trufflehog git file://.`

¿Repositorio privado = seguro?

- Ayuda, pero no basta: cualquiera con acceso al repo ve la clave.
- Si alguna vez fue público, aunque sea un día, trata la clave como expuesta.
- Las claves de producción van en las variables del hosting (Vercel, Netlify, Railway), no en el código.

PARTE 03

POR SERVICIO

No todas las claves son secretas. El problema es poner la secreta donde va la pública, o dejar la pública sin reglas.

BASE DE DATOS

Supabase

- ✓ Navegador: `anon` o publishable key
- ✕ Nunca: `service_role` o secret key

RLS activado en todas las tablas, con políticas. Sin RLS, la anon key puede leer y borrar todo.

BASE DE DATOS

Firebase

- ✓ Navegador: la config del proyecto
- ✕ Nunca: la cuenta de servicio (JSON del Admin SDK)

Tus reglas no dicen `if true`. Nada de «modo de prueba» en producción.

PAGOS

Stripe

- ✓ Navegador: `pk_live_`
- ✕ Nunca: `sk_live_` ni `whsec_`

Los cobros y webhooks se crean en el servidor. El precio nunca lo manda el navegador.

INTELIGENCIA ARTIFICIAL

OpenAI, Claude, Gemini

- ✓ Navegador: ninguna
- ✕ Nunca: `sk-`, `sk-ant-` ni ninguna otra

Las llamadas pasan por tu backend o una edge function, con usuario autenticado.

MAPAS

Google Maps

- ✓ Navegador: la API key, restringida
- ✕ Nunca: una clave sin restricciones

En Google Cloud: limitada a tu dominio y solo a las APIs que usas.

REGLA DE ORO

Si cobra o lee datos, va en el servidor

Si una clave puede gastar dinero o leer datos privados, el navegador no la ve nunca. Ni con un prefijo público ni «solo por ahora».

PARTE 04

PROTECCIONES EXTRA

Aunque todo lo anterior esté bien, esto limita el daño si algo se escapa.

- [ ] **Límite de gasto** en cada API de pago (OpenAI, Anthropic, Google Cloud) y alerta por correo al 50% y al 80%.
- [ ] **Límite de peticiones** en tus endpoints de IA, por usuario y por IP. Si no, alguien puede usar tu backend gratis.
- [ ] Los endpoints que gastan dinero piden **usuario con sesión iniciada**.
- [ ] Tus **logs y mensajes de error** no muestran claves, tokens ni el `.env`.
- [ ] Una clave para **desarrollo** y otra para **producción**, cada una con los permisos mínimos.
- [ ] Sabes **quién tiene acceso** a tus paneles (hosting, base de datos, pagos) y quitaste a quien ya no trabaja contigo.

Los 3 errores más comunes en apps hechas con IA

Clave de IA en el frontend

La app llama a OpenAI o Claude desde el navegador, con la clave a la vista.

Supabase sin RLS

Las tablas no tienen reglas, así que la clave pública puede leer y borrar todo.

Firebase en modo de prueba

Las reglas siguen abiertas a cualquiera, también en producción.

PARTE 05

SI ENCONTRASTE UNA CLAVE

Borrarla del código no basta: da por hecho que ya la copiaron. Hazlo en este orden.

1. **Rota la clave**
   Crea una nueva en el panel del servicio y revoca la vieja. Este es el paso que de verdad te protege.
2. **Guárdala bien**
   Pon la nueva solo en el servidor o en las variables del hosting.
3. **Mueve la llamada**
   Pasa la llamada al backend o a una edge function, para que el navegador nunca vea la clave.
4. **Revisa el uso**
   Mira el consumo y la facturación de los últimos días por si alguien la usó.
5. **Limpia el historial**
   Si estaba en GitHub, bórrala del historial con `git filter-repo` o BFG.
6. **Revisa los datos**
   Si había datos de usuarios expuestos, revisa qué se pudo leer y avisa a quien corresponda.

Lo que no te protege

- Borrar la clave del código sin rotarla.
- Ocultarla en otra variable o «cifrarla» en el frontend.
- Poner el repositorio privado después de que fue público.

PARTE 06

¿LO REVISAMOS JUNTOS?

Puedes hacer todo esto por tu cuenta con este checklist. Si prefieres que lo vea yo, hacemos una llamada y revisamos tu sistema juntos.

SEGURIDAD

Una capa de seguridad para tu app

Revisamos tu app y te digo qué está expuesto y cómo cerrarlo.

- Dónde están tus claves y qué se ve desde el navegador.
- Llamadas a IA y pagos movidas al servidor.
- Reglas de tu base de datos: RLS en Supabase o reglas de Firebase.
- Límites de gasto y de peticiones para que nadie use tu saldo.

Ideal si hiciste tu app con IA y no sabes qué tan expuesta está.

FUNCIONALIDAD

Si quieres, también la mejoramos

Aprovechamos la llamada para ver qué puede funcionar mejor.

- Qué falla, qué va lento y por qué.
- Qué le falta para tus usuarios o tu negocio.
- Automatizaciones y dashboards que te ahorren trabajo.
- Un plan claro con lo que conviene hacer primero.

Opcional: solo si lo quieres.

1 · Escríbeme

Mándame «LLAMADA» y el link de tu app.

2 · Agendamos

Buscamos un horario y me cuentas qué hace tu app.

3 · La revisamos

En la llamada vemos tu sistema y qué conviene arreglar.

ESCRÍBEME "LLAMADA"

Mándame la palabra por mensaje directo con el link de tu app y agendamos la llamada.

Instagram **@ssseleniadev** TikTok **@seleedev**

De esto y de cómo programar en la era de la IA hablé en el podcast **Se Vende Humo**, episodio 62. Búscalo en Spotify.